# PTR5932 - Introdução à Ciência de Dados Geoespaciais

Diego Bogado Tomasiello

2026-09-18

# Table of contents


**Apresentação** **4**
Organização do material . . . . . 4

**1 Aula 01** **6**
1.1 Interface do RStudio . . . . . 6
1.1.1 Painel de Scripts . . . . . 7
1.1.2 Painel de Ambiente . . . . . 7
1.1.3 Painel Console . . . . . 7
1.1.4 Painel de Saída . . . . . 7
1.2 Regras básicas da sintaxe do R . . . . . 7
1.2.1 Como atribuir valores a variáveis . . . . . 7
1.2.2 Sensibilidade a maiúsculas e minúsculas . . . . . 8
1.2.3 Como comentar códigos . . . . . 8
1.3 Instalação de pacotes . . . . . 8
1.4 Referências: . . . . . 9

**2 Aula 02** **10**
2.1 **Sistemas de Referência de Coordenadas (CRS), transformação entre sistemas de referencia e reprojeções em R** . . . . . 10
2.1.1 **Objetivos de Aprendizagem** . . . . . 10
2.1.2 **Roteiro de Execução Passo a Passo** . . . . . 10
2.2 Referências: . . . . . 13

**3 Aula 03** **14**
3.1 **Leitura, filtragem, relacionamento, criação de novos campos e representação de dados vetoriais e matriciais** . . . . . 14
3.1.1 **Objetivos de Aprendizagem** . . . . . 14
3.1.2 **Preparação do Ambiente no R** . . . . . 14
3.1.3 **Roteiro de Execução Passo a Passo** . . . . . 15
3.2 Referências: . . . . . 16

**4 Aula 04** **17**
4.1 **Acesso e leitura de dados da INDE, Censos, OpenStreetMap, Geocodificação, Rastreamento e LiDAR** . . . . . 17
4.1.1 **Objetivos de Aprendizagem** . . . . . 17
4.1.2 **Preparação do Ambiente no R** . . . . . 17

4.1.3 **Roteiro de Execução Passo a Passo** . . . 18
4.2 Referências: . . . 24

**5 Aula 05** **25**
5.1 **Objetivos de Aprendizagem** . . . 25
5.2 **Preparação do Ambiente no R** . . . 25
5.3 **Roteiro de Execução Passo a Passo** . . . 26
5.3.1 **Operação de Buffer** . . . 26
5.3.2 **Operação de Área de Serviço** . . . 27
5.3.3 **Operações de sobreposição** . . . 28
5.3.4 **Spatial Join** . . . 29
5.4 Referências: . . . 30

**6 Aula 06** **32**
6.1 **Objetivos de Aprendizagem** . . . 32
6.2 **Preparação do Ambiente no R** . . . 32
6.3 **Roteiro de Execução Passo a Passo** . . . 33
6.3.1 **Álgebra de mapas** . . . 33
6.3.2 **Estatística Zonal** . . . 34
6.3.3 Extração de valores de dados raster a partir de pontos . . . 36
6.3.4 **Reclassificação de dados Raster** . . . 37
6.4 Referências: . . . 38

**Referências** **39**

# Apresentação

Autor: Diego Bogado Tomasiello

Data: 18/09/2026

Vivemos em uma era de abundância de dados sem precedentes. O avanço acelerado das tecnologias de informação e comunicação transformou a forma como registramos, analisamos e compreendemos os fenômenos ao nosso redor, gerando um volume maciço de informações que trazem consigo um elemento fundamental: a localização geográfica. No planejamento, na operação e na gestão dos sistemas de transportes e da infraestrutura urbana, essa dimensão espacial pode permitir desvendar padrões complexos da dinâmica territorial.

No entanto, a mera disponibilidade de dados geoespaciais não garante sua utilidade. Um dos desafios na academia e no mundo corporativo é a capacidade de transformar grandes volumes de dados geoespaciais em conhecimento estruturado. É nesse cenário desafiador que se insere este livro, concebido como o material didático de apoio para a disciplina PTR-5932: Introdução à Ciência de Dados Geoespaciais, do Programa de Pós-Graduação em Engenharia de Transportes da Escola Politécnica da Universidade de São Paulo (USP).

Este material foi estruturado especificamente para preencher uma lacuna: a necessidade de uma formação introdutória que forneça fundamentos teóricos e instrumentais para a organização, manipulação e representação de dados geoespaciais. Destinado a estudantes de mestrado, doutorado, pesquisadores e profissionais de áreas correlatas, o material tem como objetivo construir uma base conceitual que dê segurança e autonomia para que, em etapas posteriores de suas trajetórias, os leitores possam avançar para a construção de modelos e análises espaciais.

Para atingir esse objetivo, o material apresenta laboratórios didáticos com aplicações na linguagem de programação R.

## Organização do material

| Aula | Descrição |
|---|---|
| 1. Introdução | Instalação do R e do RStudio. Apresentação da interface do RStudio, sintaxe da linguagem e instalação de pacotes. |
| 2. Sistemas de Referência e Projeções Cartográficas | Transformação entre Sistemas de Referência e Projeções Cartográficas. |
| 3. Dados espaciais: tipos, estruturas, formas de representação e formatos de armazenamento | Leitura de diferentes formatos de arquivo no R |
| 4. Dados de entrada em SIG: Infraestruturas de Dados Espaciais, Censos, Pesquisas Origem e Destino, VGI, Geocodificação, GNSS etc. | Pacotes para leitura de diferentes formatos de dados, tratamento topológico e geocodificação de dados. |
| 5. Operações espaciais: buffer, clip, dissolve, intersect, union etc. | Principais operações de sobreposição e proximidade |
| 6. Manipulação de dados raster: álgebra de mapas | Leitura, escrita e operações espaciais (locais, focais, zonais e globais) com dados raster |
| 7. Modelagem de redes de transportes: redes em SIG e em R (r5r) | Princípios de modelagem de redes de transporte em SIG e em R usando o pacote r5r |
| 8. Design e produção de mapas | Produção de mapas utilizando o pacote cowplot |

# 1 Aula 01

Pré-requisitos:

- Instalar o R
- Instalar o RStudio

Instalar os pacotes: - `sf` - `terra` - `dplyr` - `data.table` - `mapview` - `osmdata` - `geobr` - `geocodebr`

## 1.1 Interface do RStudio

Após instalar, em sequencia, o R e o RStudio, abra o RStudio e observe a sua interface (Figura 01).

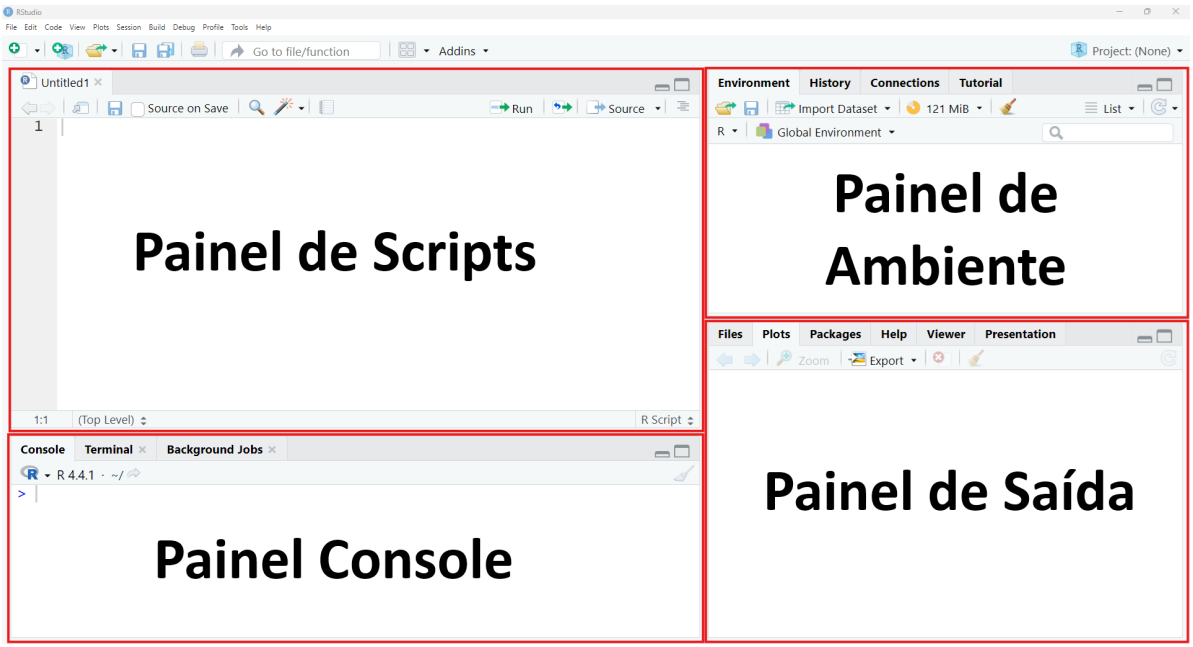



Figure 1.1: Figura 01 - Interface do RStudio

### 1.1.1 Painel de Scripts

Painel onde você pode editar e salvar scripts em R ou criar documentos computacionais, como Quarto e R Markdown.

### 1.1.2 Painel de Ambiente

Painel que exibe os objetos temporários em R à medida que são criados durante a sessão do R.

### 1.1.3 Painel Console

Painel utilizado para escrever comandos curtos e interativos em R.

### 1.1.4 Painel de Saída

Painel que exibe os gráficos, tabelas ou resultados em HTML gerados pelo código executado, juntamente com os arquivos salvos em disco.

## 1.2 Regras básicas da sintaxe do R

A seguir são apresentadas as regras básicas da sintaxe da linguagem R

### 1.2.1 Como atribuir valores a variáveis

A atribuição de um valor a uma variável é feita através dos operadores `<-` ou `=`:

```
x <- 10
x = 10
```

### 1.2.2 Sensibilidade a maiúsculas e minúsculas

O R diferencia atributos e nomes de variáveis em letras maiúsculas e minúsculas:

```
Altura <- 170
altura <- 180

print(Altura)
```

```
[1] 170
```

```
print(altura)
```

```
[1] 180
```

```
"SP" == "sp"
```

```
[1] FALSE
```

### 1.2.3 Como comentar códigos

Os comentários servem como anotações para organizar o código e torná-lo mais fácil de compreender e replicar. Para inserir um comentário, utilize o operador `#`. Esse operador pode ser utilizado como cabeçalho para descrever blocos de código, no início de uma linha para evitar que o código seja executado ou ao final de uma linha de código. Para comentar um bloco inteiro de código, selecione as linhas desejadas e pressione `Crtl` + `Shift` + `C`:

```
# Vamos agora atribuir o valor 170 para a variável Altura
Altura <- 170

altura <- 180 # Agora atribuímos o valor 180 para a variável altura

# print(altura) # dessa forma evitamos a impressão do valor da variável.
```

## 1.3 Instalação de pacotes

Para instalar pacotes no RStudio, é utilizada a função `install.packages()` no console do RStudio:

```
install.packages()
```

Para instalar diversos pacotes de uma única vez, digite no console do RStudio a função `install.packages ()` utilizando como argumento os pacotes `sf`, `terra`, `dplyr`, `data.table`, `mapview`, `osmdata`, `geobr` e `geocodebr`, que são alguns dos pacotes que serão utilizados ao longo da disciplina:

```
install.packages(c("sf", "terra", "dplyr", "data.table", "mapview", "osmdata", "geobr", "geo
```

Após instalar os pacotes, carregue-os na sessão atual através da função `library()`:

```
library(sf)
```

## 1.4 Referências:

https://docs.posit.co/ide/user/

# 2 Aula 02

A compatibilização dos sistemas de referencia e das projeções cartográficas é fundamental para garantir a correta integração de diferentes dados geoespaciais. Este laboratório didático tem como objetivo exercitar a identificação de **Sistemas de Referência de Coordenadas (CRS)**, a **transformação entre sistemas de referencia e a reprojeções de dados geoespaciais**.

## 2.1 Sistemas de Referência de Coordenadas (CRS), transformação entre sistemas de referencia e reprojeções em R

### 2.1.1 Objetivos de Aprendizagem

- Inspecionar parâmetros dos CRS (Datum, Elipsoide, Unidade e Projeção) via código **EPSG**
- Identificar incompatibilidade de CRS entre diferentes fontes de dados vetoriais.
- Reprojetar com a função `st_transform()`.
- Atribuir CRS com a função `st_set_crs()`.

### 2.1.2 Roteiro de Execução Passo a Passo

#### 2.1.2.1 Preparação do Ambiente no R

```
# Carregar as bibliotecas fundamentais
library(sf)
library(dplyr)
```

#### 2.1.2.2 Leitura do Dado e Inspeção do CRS Original

Baixe os arquivos disponibilizados via Google Classroom e configure o diretório de trabalho utilizando a função `setwd()`. Leia o arquivo `zonas_origem_destino.gpkg` através da função `read_sf()` e inspecione seu CRS por meio da função `st_crs()`:

```
# Configuração do diretório de trabalho
setwd() # Ajustar para a pasta onde estão localizadas as bases de dados. (p.ex. "C:/Users/Use

# Leitura da camada vetorial (GeoPackage)
zonas_sp <- read_sf("zonas_origem_destino.gpkg")

# Inspeção do CRS da camada
st_crs(zonas_sp)

# Extração direta do código EPSG e das unidades de medida planimétricas
epsg_original <- st_crs(zonas_sp)$epsg
unidade_medida <- st_crs(zonas_sp)$units_gdal
cat("Código EPSG detectado:", epsg_original, "\n")
cat("Unidade de medida das coordenadas:", unidade_medida, "\n")
```

> **Nota:** A camada está em **WGS84 Sistema de Coordenadas Geográficas (EPSG: 4326)**, um dos sistemas de referencia mais utilizados em dados geoespaciais. O WGS84 é o sistema de referência utilizado pelo sistema de posicionamento GPS. Nos sistemas de coordenadas geográficas, as coordenadas são expressas em *graus decimais*.

#### 2.1.2.3 Incompatibilidade de CRS

Carregue a segunda camada (`estacoes_metro.gpkg`) e verifique a compatibilidade entre os CRS:

```
estacoes <- read_sf("estacoes_metro.gpkg")

# Teste lógico de igualdade entre os sistemas de referência
if (st_crs(zonas_sp) != st_crs(estacoes)) {message("As camadas possuem CRS distintos e não de
```

> **Nota:** A camada está em **Córrego Alegre UTM 23S (EPSG: 22523)**, um sistema de referência geodésico local utilizado históricamente no Brasil e frequentemente encontrado em dados cartográficos mais antigos, especialmente extraídos a partir de cartas topográficas digitalizadas. O Córrego Alegre foi posteriormente substituído pelo SAD69 e, mais recentemente, pelo SIRGAS2000. Como a projeção

cartográfica adotada é a Universal Transversa de Mercator (UTM), as coordenadas são expressas em *metros*.

#### 2.1.2.4 Transformação Espacial (Reprojeção)

Para a Região Metropolitana de São Paulo, adotaremos o sistema de referência projetado **SIRGAS 2000 / UTM Fuso 23S (EPSG: 31983)**, que utiliza unidades métricas. Para reprojetar os dados, utilize a função `st_transform()`, informando como parâmetros a variável de entrada e o código EPSG do sistema de referência, `31983`:

```
# Reprojeção da camada de Zonas de Tráfego para SIRGAS 2000 / UTM Fuso 23S
zonas_utm <- st_transform(zonas_sp, crs = 31983)

# Reprojeção da camada de Estações para SIRGAS 2000 / UTM Fuso 23S
estacoes_utm <- st_transform(estacoes, crs = 31983)

# Confirmação do novo CRS
st_crs(zonas_utm)$Name # Retorno esperado: "SIRGAS 2000 / UTM zone 23S"

# Confirmação do novo CRS
st_crs(estacoes_utm)$Name # Retorno esperado: "SIRGAS 2000 / UTM zone 23S"
```

#### 2.1.2.5 Atribuição de um sistema de referência e projeção

Caso a camada de entrada não possua um sistema de referência e projeção, é possível atribuir o CRS adequado à camada, desde que essa informação seja conhecida previamente, utilizando a função `st_set_crs()`:

```
# Leitura da camada de linhas do Metrô de São Paulo
linhas_metro <- read_sf("linhas_metro_SIRGAS2000_UTM23S.gpkg")

# Consulta do CRS da camada
st_crs(linhas_metro) #Retorno esperado: NA

# Atribuição do sistema de referência e projeção corretos (baseado no nome da camada)
linhas_metro <- linhas_metro %>%
  st_set_crs(31983)

# Consulta do CRS da camada
st_crs(linhas_metro)$Name #Retorno esperado: SIRGAS 2000 / UTM zone 23S
```

## 2.2 Referências:

https://r-spatial.github.io/sf

https://cran.r-project.org/web/packages/sf/index.html

https://r.geocompx.org/reproj-geo-data

# 3 Aula 03

Os formatos de representação vetorial e matricial são os mais utilizados para modelar a realidade. Este laboratório didático possui como objetivo exercitar a **leitura, filtragem, relacionamento, criação de novos campos e a representação de dados vetoriais e matriciais.**

## 3.1 Leitura, filtragem, relacionamento, criação de novos campos e representação de dados vetoriais e matriciais

### 3.1.1 Objetivos de Aprendizagem

- Exercitar a leitura e a filtragem de dados vetoriais através das funções `read_sf()` e `filter()`.
- Relacionar a tabela de atributos de um dado vetorial com um tabular através da função `left_join()`.
- Criar um novo campo com operações matemáticas de campos já existentes através da função `mutate()` e visualizar o dado vetorial através da função `mapview()`.
- Rasterizar um dado vetorial através da função `rasterize()` e visualizá-lo através da função `plot()`.
- Exportar dados vetoriais e raster, respectivamente, através das funções `write_sf()` e `writeRaster()`.

### 3.1.2 Preparação do Ambiente no R

```
# Carregar as bibliotecas fundamentais
library(sf)
library(terra)
library(dplyr)
library(data.table)
library(mapview)
```

### 3.1.3 Roteiro de Execução Passo a Passo

#### 3.1.3.1 Leitura e filtragem da base de dados vetorial

Baixe os arquivos disponibilizados via Google Classroom e configure o diretório de trabalho através da função `setwd()`. Leia o arquivo `zonas_origem_destino.gpkg` utilizando a função `read_sf()` e filtre somente as zonas OD pertencentes ao município de São Paulo (código do município == 36) através da função `filter()`:

```
# Configure o diretório de trabalho
setwd() # Ajustar para a pasta onde estão localizadas as bases de dados. (p.ex. "C:/Users/Us

# Leia a camada vetorial (GeoPackage) e filtre somente as zonas OD do município de São Paulo
zonas_od <- read_sf("zonas_origem_destino.gpkg") %>%
  filter(cd_municipio == 36)
```

> **Nota:** O comando `%>%` e a função `filter()` só funcionarão se a biblioteca `dplyr` tiver sido carregada.

#### 3.1.3.2 Leitura do arquivo tabular e relacionamento de tabela

Leia o arquivo `tabela01.csv` através da função `fread()`. Em seguida, relacione o dado tabular com o das zonas OD através da função `left_join()`, crie um novo campo com o valor da densidade de domicilios por hectare através da função `mutate()` e visualize os resultados através da função `mapview()`:

```
#Leia o dado tabular
tabela01 <- fread("Tabela01.csv", encoding = "Latin-1")

# Relacione a tabela preservando a classe espacial 'sf'
zonas_od_join <- zonas_od %>%
  left_join(tabela01, by = c("cd_original_zona_origem_destino"="Zona"))

# Crie um campo nomeado "dens_domicilios" que recebe os valores da divisão dos campos "Domici
zonas_od_join <- zonas_od_join %>%
  mutate(dens_domicilios = Domicilios/Area_ha)

# Mude o mapa base da função mapview para imagens de satélite
mapviewOptions(basemaps = c("Esri.WorldImagery", "OpenTopoMap", "OpenStreetMap"))

# Visualize a variável de domicílios através da função mapview
mapview(zonas_od_join, zcol = "dens_domicilios")
```

**Nota:** Como o dado tabular possui acentos e caracteres especiais, foi adotado o parâmetro de `encoding = Latin-1` dentro da função `fread()`.

#### 3.1.3.3 Rasterização da camada vetorial

Para rasterizar dados vetoriais, utilize as funções `rast()` e `rasterize()`. Em seguida, visualize o arquivo raster através da função `plot()`:

```
# crie o arquivo raster de template
raster_template <- rast(zonas_od_join, resolution = 0.00009)

# Execute a rasterização definindo o atributo numérico de interesse
raster_resultado <- rasterize( x = zonas_od_join, y = raster_template, field = "dens_domicili

# Visualize o raster gerado
plot(raster_resultado, main = "Densidade de domicílios por hectare")
```

#### 3.1.3.4 Exporte arquivos vetoriais e raster

Exporte arquivos vetoriais e raster, respectivamente, através das funções `write_sf()` e `writeRaster()`:

```
# Exporte o dado vetorial utilizando a função write_sf()
write_sf(zonas_od_join, "zonas_od_join.gpkg")

# Exporte o dado raster utilizando a função writeRaster()
writeRaster(x = raster_resultado, filename = "densidade_domicilios.tif", filetype = "GTiff",
```

## 3.2 Referências:


https://r-spatial.github.io/sf

https://cran.r-project.org/web/packages/terra/refman/terra.html

# 4 Aula 04

A etapa de entrada de dados é fundamental para qualquer projeto em Ciência de Dados Geoespaciais. A presente aula aborda como importar algumas das principais fontes de dados espaciais. O conteúdo abrange análise de topologia e o acesso a bases de dados como a **Infraestrutura Nacional de Dados Espaciais (INDE)**, dados de **Censos**, **Geocodificação**, **OpenStreetMap**, **Rastreamento** e **LiDAR**.

## 4.1 Acesso e leitura de dados da INDE, Censos, OpenStreetMap, Geocodificação, Rastreamento e LiDAR

### 4.1.1 Objetivos de Aprendizagem

- Identificar e corrigir erros topológicos.
- Acessar dados da Infraestrutura Nacional de Dados Espaciais (INDE).
- Acessar dados do Censo através dos pacotes `censobr` e `geobr`.
- Geocodificar uma lista de endereços através do pacote `geocodebr`.
- Acessar dados do OpenStreetMap através do pacote `osmdata`.
- Ler e tratar dados de rastreamento.
- Ler e tratar dados LiDAR através dos pacotes `lidR` e `terra`

### 4.1.2 Preparação do Ambiente no R

```
# Carregar as bibliotecas fundamentais
library(sf)          # Manipulação de dados vetoriais (Simple Features)
library(tidyverse)   # Manipulação e visualização de dados (dplyr, ggplot2)
library(geobr)       # Malhas territoriais oficiais do IBGE
library(censobr)     # Microdados e agregados censitários do IBGE
library(geocodebr)   # Geocodificação local com base no CNEFE/IBGE
library(lwgeom)      # funções para análise topológica
```

```r
library(osmdata)     # Consulta a dados VGI (OpenStreetMap via Overpass API)
library(mapview)     # Cartografia temática interativa
library(rstudioapi)  # Apontar o diretório de trabalho
library(data.table)  # Leitura de dados tabulares
library(lidR)        # Leitura de dados LiDAR (LAS/LAZ)
library(terra)       # Leitura, tratamento e exportação de dados raster
library(rgl)         # Renderização e visualização 3D

#Caso não tenha instalado algum dos pacotes anteriores, execute:
#install.packages("nome do pacote")

# Configure o diretório de trabalho
path <- selectDirectory(caption = "Select Working Directory") # rode esta linha para apontar

setwd(path)
mapviewOptions(basemaps = c("Esri.WorldImagery", "OpenTopoMap", "OpenStreetMap"))
```

> **Nota:** O comando `selectDirectory()` abre uma interface para que o usuário indique o diretório de trabalho.

### 4.1.3 Roteiro de Execução Passo a Passo

#### 4.1.3.1 Análise topológica das Zonas OD 2023 de São Paulo

Quando trabalhamos com dados geoespaciais vetoriais, é essencial verificarmos a sua consistência topológica. Inconsistências topológicas, como auto-intersecções (*self-intersections*), sobreposições indevidas (*overlaps*) e lacunas não planejadas (*gaps*) entre polígonos adjacentes, podem comprometer a precisão de análises territoriais, distorcer cálculos de área e provocar falhas na execução de operações espaciais. O script abaixo utiliza os pacotes `sf` e `lwgeom`, para identificar e corrigir automaticamente falhas topológicas em camadas vetoriais de polígonos:

```r
# Ler arquivo shapefile das Zonas OD 2023 disponibilizado pelo Metrô
zonas_od <- read_sf("Zonas_2023.shp", options = "ENCODING=LATIN1")

# Verificar se existem geometrias inválidas
zonas_od$valido <- st_is_valid(zonas_od)

# Contar de geometrias válidas e inválidas
table(zonas_od$valido)

# Identificar o motivo exato de cada invalidade
```

```r
zonas_od$motivo_erro <- st_is_valid(zonas_od, reason = TRUE)

# Listar os tipos de erros encontrados no dataset e armazená-los em uma variável. Inspecionar
erros <- zonas_od %>%
  filter(!valido) %>%
  st_drop_geometry() %>%
  count(motivo_erro, sort = TRUE)

# Aplicar a correção automáticamente
zonas_od_corrigidas <- st_make_valid(zonas_od)

# Verificar se ainda são encotradas geometrias inválidas
zonas_od_corrigidas$valido <- st_is_valid(zonas_od_corrigidas)

# Contar geometrias válidas e inválidas
table(zonas_od_corrigidas$valido)

# Visualizar as Zonas OD de 2023
mapview(zonas_od_corrigidas)
```

> **Nota:** Os erros mais comuns, do tipo "Self-intersection", ocorrem quando a linha de contorno da zona cruza consigo mesma formando um laço.

#### 4.1.3.2 Acesso a Infraestrutura Nacional de Dados Espaciais (INDE)

A INDE concentra o acesso a diversas bases de dados oficiais de orgãos do governo. No exemplo abaixo, acessaremos o serviço de WFS (Web Feature Service), por onde são disponibilizados dados vetoriais. Para isso, indique a URL do serviço WFS, liste as camadas existentes e faça a leitura da camada de arruamento utilizando a função `st_read()`. Em seguida, inspecione o sistema de referência e a projeção cartográfica da base de dados de arruamento e, caso seja necessário, faça os devidos ajustes. Por fim, visualize a base de dados através da função `mapview()`, diferenciando o arruamento de acordo com seu revestimento:

```r
# Consumo de uma camada WFS (Web Feature Service) da INDE
url_wfs <- sprintf("https://geoservicos.inde.gov.br/geoserver/wfs") # Exemplo conceitual de

#info_camadas <- st_layers(paste0("WFS:", url_wfs)) # Lista as camadas do WFS, porém para a

# Leitura direta de camada espacial vetorial via protocolo OGC WFS sem download manual
# sf permite conexão direta com URLs WFS/GeoJSON da INDE
IDE_camada <- st_read(dsn = paste0(url_wfs, "?service=WFS&version=1.0.0&request=GetFeature&ty
```

```r
# Inspeção do Sistema de Referência de Coordenadas (CRS / SIRGAS 2000 - EPSG:4674)
st_crs(IDE_camada)

IDE_camada <- st_set_crs(IDE_camada, 4674)
st_crs(IDE_camada)

mapviewOptions(basemaps = c("Esri.WorldImagery", "OpenTopoMap", "OpenStreetMap"))

mapview::mapview(IDE_camada, zcol = "revestimen")
```

**Nota:** Foi atribuído o EPSG 4674, pois todos os dados disponibilizados através da INDE estão no referencial SIRGAS2000 e em Sistemas de Coordenadas Geográficas.

#### 4.1.3.3 Acesso a dados do Censo Demográfico

Serão utilizados os pacotes `geobr` e `censobr` para baixar, respectivamente, os dados espaciais e os dados tabulares dos setores censitários. Em seguida, os dados são relacionados através da função `left_join()` e visualizados através da função `mapview()`:

```r
# Download da malha de setores censitários de um município (ex: Armação de Búzios/RJ) via ge
setores_ab <- read_census_tract(year = 2022, code_tract = 3300233)
st_crs(setores_ab)

# Leitura dos dados do Censo 2022 agregados por Setor Censitário via censobr
dados_setores <- read_tracts(year = 2022, dataset = "Basico") %>%
  filter(code_muni == 3300233) %>% # filtra apenas o município de Armação de Búzios/RJ
  collect() # Traz para a memória o resultado filtrado

data_dictionary(year = 2022, dataset = "tracts") # dicionario de dados para verificar os nom

dados_setores <- dados_setores %>%
  select(code_tract, V0001) # seleciona apenas as variaveis de código do setor (code_tract)

setores_ab_join <- setores_ab %>%
  left_join(dados_setores, by="code_tract") #join do dado espacial de setores censitários co

# Visualizacao dos dados de população por setor censitário de 2022
mapview(setores_ab_join, zcol = "V0001")
```

**Nota:** Na função `read_tracts()` foi selecionado o dataset "Basico". No entanto, existem ainda os datasets "Pessoas", "Domicilios", "Obitos", "Indigenas", "Quilom-

bolas", "ResponsavelRenda" e "Entorno", ou seja, existem muitos outros dados a serem explorados.

#### 4.1.3.4 Geocodificação

A geocodificação consiste em transformar um endereço (ou lista de endereços) em um par (ou vários pares) de coordenadas. A geocodificação pode ser realizada gratuitamente através do pacote `geocodebr`:

```
# Leitura da lista de endereços para geocodificação
enderecos <- fread("RMSPEstb_2014_10000reg.csv")

# Mapeamento dos campos com definir_campos()
campos <- definir_campos(
  logradouro = "nomelogradouro",
  numero = "numero",
  municipio = "municipio",
  estado = "uf")

# 3. Execução da geocodificação via geocodebr
resultados_geocod <- geocode(
  enderecos = enderecos,
  campos_endereco = campos
)

# O resultado retorna com níveis de precisão da geocodificação, longitude e latitude
head(resultados_geocod)

# Espacialização dos resultados
resultados_geocod_sf <- st_as_sf(resultados_geocod, coords = c("lon", "lat"), crs = 4674)

# Visualização dos resultados de acordo com a precisão da geocodificação
mapview(resultados_geocod_sf, zcol = "precisao")
```

#### 4.1.3.5 Acesso aos dados do OSM

Uma fonte importante de dados espaciais são os mapeamentos colaborativos. Na área de transportes, uma das principais iniciativas é o OpenStreetMap (OSM), que disponibiliza informações sobre a rede viária em escala mundial. Os dados do OSM podem ser acessados através do pacote `osmdata`:

```r
# Definição da caixa delimitadora (Bounding Box)
bb <- opq(bbox = "São Paulo, Brasil")

# Consulta à API Overpass para extrair ciclovias (cycleways)
ciclovias <- bb %>%
  add_osm_feature(key = "highway", value = "cycleway") %>%
  osmdata_sf()

# Extração das geometrias do tipo linha
linhas_ciclovias <- ciclovias$osm_lines

# Visualização das ciclovias classificadas pelo atributo de segregação (parâmetro zcol)
mapviewOptions(basemaps = c("Esri.WorldImagery", "OpenTopoMap", "OpenStreetMap"))
mapview(linhas_ciclovias, zcol = "segregated")
```

#### 4.1.3.6 Dados de rastreamento

Outra fonte abundante de dados espaciais, principalmente na área de transportes, são os dados de rastreamento. Esses dados são normalmente obtidos a partir do rastreamento de receptores GNSS, que podem estar acoplados a veículos ou a qualquer outro objeto de interesse. Os dados de rastreamento podem apresentar formatos heterogêneos, mas precisam conter, no mínimo, latitude, longitude, data (dia, mês e ano) e horário do registro. A seguir, apresenta-se um exemplo de espacialização de uma base de dados de rastreamento de ônibus:

```r
# Leitura da base de dados de rastreamento de ônibus da SPTrans (Linha 34831)
 rastreio <- fread("rastreio_linha34831_data_01_03_2015.csv")

# Dicionario de dados:
 #V1 = 'dataserver'
 #V2 = 'dataavl'
 #V3 = 'linha'
 #V4 = 'lat'
 #V5 = 'lon'
 #V6 = 'codavl'
 #V7 = 'desc1'
 #V8 = 'desc2'

# Espacialização da base de dados de rastreamento
rastreio_espacializado <- rastreio %>%
  st_as_sf(coords = c("V5", "V4"), crs = 4326)
```

```r
# Visualização da base de dados
mapview(rastreio_espacializado)
```

#### 4.1.3.7 Dados LiDAR

Por fim, dados espaciais provenientes de levantamentos com sensores LiDAR tem sido encontrados cada vez mais frequentemente em estudos urbanos. Os sensores LiDAR podem ser acoplados a diferentes tipos de plataformas e constituem uma fonte rica de dados para extrair informações sobre a cobertura da superfície e gerar Modelos Digitais de Elevação. Os dados LiDAR podem ser lidos e processados através dos pacotes `lidR`, `rgl` e `terra`:

```r
# Ler o arquivo LAZ
las <- readLAS("MDS_3313-152_1000.laz")

# Validar e inspecionar os metadados da nuvem (densidade de pontos, atributos)
print(las)

# Checar a integridade da geometria da nuvem
las_check(las)

# Visualização 3D da Nuvem de Pontos Original
plot(las, color = "Z", bg = "black", axis = TRUE)

# Filtragem dos pontos classificados como Terreno (Padrão ASPRS: Classe 2 = Terreno).
las_terreno <- filter_ground(las)

# Visualizar apenas a nuvem filtrada do solo em 3D
plot(las_terreno, color = "Z", bg = "white")

# Geração do Modelo Digital do Terreno (MDT / DTM)
mdt <- rasterize_terrain(
  las = las_terreno,
  res = 0.5, # Resolução espacial da célula (ex: 1 metro)
  algorithm = tin() # Algoritmo de interpolação por Triangulação Delaunay
  )

# Plotagem 2D do MDT gerado (objeto SpatRaster do pacote 'terra')
plot(mdt, main = "Modelo Digital do Terreno (MDT) - Resolução: 1m", col = terrain.colors(50))

# Exportar o raster resultante para formato GeoTIFF
writeRaster(mdt, filename = "MDT.tif", overwrite = TRUE)
```

## 4.2 Referências:


https://r-spatial.github.io/sf

https://cran.r-project.org/web/packages/censobr/vignettes/censobr.html

https://cran.r-project.org/web/packages/geobr/index.html

https://cran.r-project.org/web/packages/geocodebr/index.html

https://cran.r-project.org/web/packages/osmdata/index.html

https://cran.r-project.org/web/packages/lidR/index.html

https://cran.r-project.org/web/packages/rgl/index.html

https://cran.r-project.org/web/packages/terra/refman/terra.html

# 5 Aula 05

As operações espaciais são funções que utilizam a geometria dos dados de entrada para realizar análises e produzir resultados derivados. Serão exploradas operações espaciais de **Buffer**, **Service Area**, **Intersect**, **Clip**, **Merge**, **Union**, **Erase** e **Symmetric Difference**.

## 5.1 Objetivos de Aprendizagem

- Aprender a utilizar as ferramentas de:
  - Buffer;
  - Service Area;
  - Intersect;
  - Clip;
  - Merge;
  - Union;
  - Erase;
  - Symmetric Difference; e
  - Spatial Join

## 5.2 Preparação do Ambiente no R

```
# Carregar as bibliotecas fundamentais
library(sf)          # Manipulação de dados vetoriais (Simple Features)
library(dodgr)       # Geração de rotas em rede
library(osmdata)     # Consulta a dados VGI (OpenStreetMap via Overpass API)
library(dplyr)       # Manipulação de dados
library(mapview)     # Cartografia temática interativa
library(rstudioapi)  # Apontar o diretório de trabalho
```

```r
library(terra)       # Leitura, tratamento e exportação de dados

#Caso não tenha instalado algum dos pacotes anteriores, execute:
#install.packages("nome do pacote")

# Configure o diretório de trabalho
path <- selectDirectory(caption = "Select Working Directory") # rode esta linha para apontar

setwd(path)
mapviewOptions(basemaps = c("Esri.WorldImagery", "OpenTopoMap", "OpenStreetMap"))
```

> **Nota:** O comando `selectDirectory()` abre uma interface para que o usuário indique o diretório de trabalho.

## 5.3 Roteiro de Execução Passo a Passo

### 5.3.1 Operação de Buffer

A operação de *buffer* é utilizada quando há necessidade de identificar áreas próximas a entes ou feições espaciais. O script abaixo utiliza o pacote `sf` para executar a operação de *buffer* em três camadas com geometrias diferentes: ponto, linha e polígono.

```r
# Leitura da camada de polígonos (tem que possuir atributos variáveis para buffer e crs métri
Pol <- read_sf("Zonas_2023.shp", options = "ENCODING=LATIN1") %>%
  st_make_valid() %>%
  st_transform(31983)

st_crs(Pol)

# Buffer Positivo
buffer_positivo <- st_buffer(Pol, dist = 1000)

# Buffer Negativo
buffer_negativo <- st_buffer(Pol, dist = -50)

# Buffer baseado nos Atributos de um Campo
buffer_por_atributo <- st_buffer(Pol, dist = Pol$NumDistrit * 10)

mapview(Pol) + mapview(buffer_positivo) + mapview(buffer_negativo) + mapview(buffer_por_atril
```

```r
# Leitura da camada de pontos (tem que possuir crs métrico)
pontos <- read_sf("geoportal_estacao_metro_v2.gpkg")
st_crs(pontos)

# Buffer Positivo
pontos_buffer_positivo <- st_buffer(pontos, dist = 500)

mapview(pontos_buffer_positivo) + mapview(pontos)


# Leitura da camada de linhas (tem que possuir crs métrico)
linhas <- read_sf("geoportal_linha_metro_v4.gpkg")
st_crs(linhas)

# Buffer Positivo
linhas_buffer_positivo <- st_buffer(linhas, dist = 500)

mapview(linhas_buffer_positivo)+ mapview(linhas)
```

> **Nota:** a operação de *buffer* negativo só pode ser executada em camadas com geometria do tipo polígono.

### 5.3.2 Operação de Área de Serviço

Na operação de Área de Serviço, em vez de distâncias euclidianas, como na operação de *buffer*, são consideradas as distâncias ao longo da rede de transporte para a análise de proximidade. A seguir, é apresentado um script para calcular a área de serviço de um ponto específico:

```r
# Leitura da camada de pontos
estacao <- pontos %>%
  filter(nm_estacao_metro_trem == "FRADIQUE COUTINHO") %>%
         st_transform(4326)

estacao_buffer <- st_buffer(estacao, dist = 500)

bbox <- st_bbox(estacao_buffer)
osm_sc <- opq(bbox = bbox, timeout = 180, memsize = 268435456)) %>%
  add_osm_feature(key = "highway") %>%
  osmdata_sc()

# Conversão dos dados OSM para um grafo roterizável
```

```r
graph <- weight_streetnet(osm_sc, wt_profile = "foot")

# Cálculo dos pontos alcançáveis dentro de 1.000 metros de distância em rede
from_vertex <- dodgr_vertices(graph)[which.min(
  st_distance(estacao,
              st_as_sf(dodgr_vertices(graph),
                       coords = c("x", "y"),
                       crs = 4326))), "id"]

distances <- dodgr_distances(graph, from = from_vertex)
accessible_nodes <- dodgr_vertices(graph)[which(distances <= 500), ]

# Conversão dos pontos acessíveis em uma Área de Serviço
service_area_points <- st_as_sf(accessible_nodes, coords = c("x", "y"), crs = 4326)
service_area_poly <- st_convex_hull(st_union(service_area_points))

mapview(estacao, col.regions = "red") + mapview(service_area_poly, col.regions = "white") + r
```

### 5.3.3 Operações de sobreposição

As operações de sobreposição estão entre as principais operações executadas no dia-a-dia de usuários de Sistemas de Informação Geográfica (SIG). No script abaixo são executadas as operações de *Intersect*, *Clip*, *Merge*, *Union*, *Erase* e *Symmetric Difference*:

```r
# Leitura da base de dados de favelas na subprefeitura de Campo limpo
favelas <- read_sf("favelas_campo_limpo.gpkg") %>%
  st_make_valid() %>%
  vect()

mapview(favelas, col.regions = "red")

# Leitura da base de dados de pontos de ônibus na subprefeitura de Campo limpo
ptos_onibus <- read_sf("pts_onibus_campo_limpo.gpkg") %>%
  st_make_valid() %>%
  vect()

ptos_onibus_buffer <- buffer(ptos_onibus, width = 200)
mapview(ptos_onibus_buffer)

# Leitura da base de dados de estacoes de metro na subprefeitura de Campo limpo
estacoes_buffer <- read_sf("estacoes_metro_campo_limpo.gpkg") %>%
```

```r
  st_make_valid() %>%
  vect()

mapview(estacoes_buffer)

# Intersect - Retorna apenas a área e os atributos que pertence a AMBAS as camadas (favelas e
intersect_favelas_ptos_onibus <- terra::intersect(favelas, ptos_onibus_buffer)
mapview(intersect_favelas_ptos_onibus, col.regions = "green")


# Clip - Geometricamente idêntico ao Intersect, mas preserva APENAS os atributos da camada qu
favelas <- terra::aggregate(terra::buffer(favelas, width = 0))
ptos_onibus <- terra::aggregate(terra::buffer(ptos_onibus_buffer, width = 0))

clip_favelas_ptos_onibus <- terra::crop(favelas, ptos_onibus)
mapview(clip_favelas_ptos_onibus, col.regions = "green")


# Merge - Junta as tabelas de atributos e geometrias em um único arquivo, mantendo os objetos
merge_estacoes_metro_ptos_onibus <- terra::union(estacoes_buffer, ptos_onibus_buffer)
mapview(merge_estacoes_metro_ptos_onibus)

# Union - Combina a geometria de ambos os objetos em uma única feição geográfica. A borda que
union_estacoes_metro_ptos_onibus <- terra::aggregate(terra::union(estacoes_buffer, ptos_onibu
mapview(union_estacoes_metro_ptos_onibus)

# Erase - Remove da camada de Favelas qualquer área que seja sobreposta pela camada de estaco
erase_favelas_estacoes_ptos_onibus <- terra::erase(favelas, union_estacoes_metro_ptos_onibus)
mapview(erase_favelas_estacoes_ptos_onibus, col.regions = "red") + mapview(union_estacoes_met

# Symmetric Difference - Retorna as áreas de Favelas e de estacoes de metro e pontos de onibu
sf_sym_diff_favelas_estacoes_ptos_onibus <- terra::symdif(favelas, union_estacoes_metro_ptos_
mapview(sf_sym_diff_favelas_estacoes_ptos_onibus)
```

### 5.3.4 Spatial Join

A operação de *Spatial Join* une os atributos de um ou mais dados de entrada baseado em um relacionamento espacial, seja ele de proximidade ou de sobreposição. O script abaixo mostra como é executada uma operação de *Spatial Join* através do R:

```r
# Leitura da base de dados de pontos de ônibus na subprefeitura de Campo limpo
ptos_onibus <- read_sf("pts_onibus_campo_limpo.gpkg") %>%
  st_make_valid() %>%
  st_transform(4326)

# Leitura das subprefeituras de São Paulo e filtro da subprefeitura de Campo Limpo
sub_campo_limpo <- read_sf("geoportal_subprefeitura_v2.gpkg") %>%
  filter(nm_subprefeitura =="CAMPO LIMPO") %>%
  st_make_valid() %>%
  st_transform(4326)

# Criação do bounding box e obtenção dos dados do OSM
bbox <- st_bbox(sub_campo_limpo)
osm_cl <- opq(bbox = bbox, timeout = 100, memsize = 10000000) %>%
  add_osm_feature(key = "highway") %>%
  osmdata_sf()

# Seleção apenas das vias do OSM
osm_vias <- osm_cl$osm_lines
st_crs(osm_vias)


# Defino um raio de tolerância (opcional)
raio_tolerancia <- 10 # medido na unidade do CRS (metros)

# Execução do Spatial Join com tolerância
join_com_tolerancia <- st_join( ptos_onibus,
                                osm_vias,
                                join = st_is_within_distance,
                                dist = raio_tolerancia,
                                left = TRUE # Mantém todos os pontos na tabela resultante
)

mapview(join_com_tolerancia, col.regions = "red") + mapview(osm_vias, color = "yellow")
```

## 5.4 Referências:


https://r-spatial.github.io/sf

https://cran.r-project.org/web/packages/osmdata/index.html

https://github.com/UrbanAnalyst/dodgr

https://cran.r-project.org/web/packages/terra/refman/terra.html

# 6 Aula 06

Os dados raster ou matriciais são comumente utilizados no geoprocessamento. Nesta aula serão exploradas operações de **álgebra de mapas** com geração de um modelo digital de superfície normalizado, **estatística zonal**, **extração de valores** de dados raster através de pontos e a **reclassificação de dados raster**.

## 6.1 Objetivos de Aprendizagem

- Aprender a realizar operações de:
  - Álgebra de mapas;
  - Estatística zonal;
  - Extração de valores de dados raster a partir de pontos; e
  - Reclassificação de dados raster.

## 6.2 Preparação do Ambiente no R

```
#instalação de pacotes específicos para a aula de hoje
install.packages(c("exactextractr", "elevatr", "stplanr"))

# Carregar as bibliotecas fundamentais
library(sf)          # Manipulação de dados vetoriais (Simple Features)
library(osmdata)     # Consulta a dados VGI (OpenStreetMap via Overpass API)
library(dplyr)       # Manipulação de dados
library(mapview)     # Cartografia temática interativa
library(rstudioapi)  # Apontar o diretório de trabalho
library(lidR)        # Leitura de dados LiDAR (LAS/LAZ)
library(terra)       # Leitura, tratamento e exportação de dados raster
library(rgl)         # Renderização e visualização 3D
library(exactextractr) # Realizar estatísticas zonais
library(elevatr)     # Acessar dados de atimetria de diferentes serviços web
```

```
library(stplanr)     # Permite realizar operações espaciais como segmentar vias
library(geobr)       # Acesso a dados oficiais do governo

# Configure o diretório de trabalho
path <- selectDirectory(caption = "Select Working Directory") # rode esta linha para apontar

setwd(path)
mapviewOptions(basemaps = c("Esri.WorldImagery", "OpenTopoMap", "OpenStreetMap"))
```

> **Nota:** O comando `selectDirectory()` abre uma interface para que o usuário indique o diretório de trabalho.

## 6.3 Roteiro de Execução Passo a Passo

### 6.3.1 Álgebra de mapas

O termo **Álgebra de mapas** é utilizado para descrever operações entre dois ou mais rasters utilizando expressões algébricas simples. No script abaixo, são gerados dois modelos digitais de elevação, sendo um modelo de superfície e outro de terreno e realizada a substração entre eles para gerar um modelo digital de superfície normalizado, ou seja, para representar apenas os elementos acima da superfície. O script utiliza os pacotes `sf` , `lidr` e `terra`.

```
# Ler o arquivo LAZ
las <- readLAS("MDS_3313-152_1000.laz")

# Visualização 3D da Nuvem de Pontos Original
plot(las, color = "Z", bg = "black", axis = TRUE)

# Filtro dos pontos classificados como Terreno (Padrão ASPRS: Classe 2 = Terreno).
las_terreno <- filter_ground(las)

# Visualizar apenas a nuvem filtrada do terreno em 3D
plot(las_terreno, color = "Z", bg = "white")

# Geração do Modelo Digital do Terreno (MDT / DTM)
mdt <- rasterize_terrain(
  las = las_terreno,
  res = 0.5, # Resolução espacial da célula (ex: 0,5 metro)
  algorithm = tin() # Algoritmo de interpolação por Triangulação Delaunay
)
```

```r
# Plotagem 2D do MDT gerado (objeto SpatRaster do pacote 'terra')
plot(mdt, main = "Modelo Digital do Terreno (MDT) - Resolução: 0,5m", col = terrain.colors(50


# Geração do Modelo Digital de Superficie (MDS / DSM)
unique(las@data$Classification)

mds <- lidR::rasterize_terrain(
  las = las,
  res = 0.5, # Resolução espacial da célula (ex: 0,5 metro)
  algorithm = tin(),  # Algoritmo de interpolação por Triangulação Delaunay
  use_class = c(1L, 2L, 3L, 4L, 5L, 6L, 8L) # classes identificadas através do comando: uniqu
  )

plot(mds, main = "Modelo Digital de Superficie (MDS) - Resolução: 0,5m", col = terrain.colors

# Geração do Modelo Digital de Superfície Normalizado (Álgebra de Mapas)
mdsn <- mds - mdt

plot(mdsn, main = "Modelo Digital de Superficie Normalizado (MDSn - Resolução: 0,5m", col = t
```

### 6.3.2 Estatística Zonal

Nas operações de estatística zonal, são utilizados dados vetoriais com geometria do tipo área para delimitar zonas sobre as quais são extraídas estatísticas de um arquivo raster. A seguir, apresenta-se um exemplo de extração de estatísticas de declividade para polígonos que representam favelas localizadas na subprefeitura de Freguesia do Ó - Brasilândia, em São Paulo. Para isso, são utilizados os pacotes `elevatr`, para obter o Modelo Digital de Elevação (MDE), e `exactextractr`, para o cálculo das estatísticas zonais:

```r
# Importar polígonos de favelas para SP (Código IBGE SP: 3550308)
favelas_sp_utm <- read_favela(year = 2022, simplified = F) %>%
  filter(code_muni == 3550308) %>%
  st_transform(31983)

mapview(favelas_sp_utm, color = "red")

# Baixar os limites das subprefeituras através do WFS do GeoSampa
url_wfs <- sprintf("http://wfs.geosampa.prefeitura.sp.gov.br/geoserver/geoportal/wfs")

#info_camadas <- st_layers(paste0("WFS:", url_wfs)) # Lista as camadas do serviço WFS do GeoS
```

```r
subprefeituras <- st_read(dsn = paste0(url_wfs, "?service=WFS&version=1.0.0&request=GetFeatu

subprefeituras <- st_set_crs(subprefeituras, 31983)

# Filtrar a subprefeitura de Freguesia-Brasilandia
subprefeitura_brasilandia <- subprefeituras %>%
  filter(nm_subprefeitura =="FREGUESIA-BRASILANDIA") %>%
  st_transform(4326)

mapview(subprefeitura_brasilandia)

# Baixar o Modelo Digital de Elevação do SRTM 30m via pacote 'elevatr'
mds_raw <- get_elev_raster(locations = subprefeitura_brasilandia, z = 11, clip = "bbox", src

# Converter para SpatRaster do pacote 'terra' e reprojetar para SIRGAS2000 projeção UTM 23S
mds <- rast(mds_raw)
mds_utm <- project(mds, "EPSG:31983")

# Verificar a resolução espacial (tamanho do pixel) do arquivo raster
res(mds_utm)

# Visualizar o MDE
plot(mds_utm, main = "Modelo Digital de Elevação (m)", col = terrain.colors(100))

# Geracao de camada de declividade (em graus) a partir do Modelo Digital de Elevação baixado
declividade <- terrain(mds_utm, v = "slope", unit = "degree", neighbors = 8)
plot(declividade, main = "Declividade em Graus (°)", col = terrain.colors(100))

# Operacao de Intersection para extrair apenas as favelas dentro dos limites da subprefeitura
subprefeitura_brasilandia_utm <- subprefeitura_brasilandia %>%
  st_transform(31983)

favelas_brasilandia_utm <- st_intersection(favelas_sp_utm, subprefeitura_brasilandia_utm)

mapview(subprefeitura_brasilandia_utm, color = "black", col.regions = "transparent") + mapvie


# Executar Estatística Zonal (Declividade Média, Máxima e Desvio Padrão por polígono de favel
favelas_brasilandia_utm$decl_media <- exact_extract(declividade, favelas_brasilandia_utm, 'me
favelas_brasilandia_utm$decl_max   <- exact_extract(declividade, favelas_brasilandia_utm, 'ma
favelas_brasilandia_utm$decl_stdev <- exact_extract(declividade, favelas_brasilandia_utm, 'st
```

```
mapview(subprefeitura_brasilandia_utm, color = "black", col.regions = "transparent") + mapvie
```

### 6.3.3 Extração de valores de dados raster a partir de pontos

A extração de valores de dados raster em localizações pontuais é uma operação comum para profissionais que trabalham com dados geoespaciais. No script a seguir, são calculadas as declividades ao longo de segmentos de vias, utilizando dados da rede viária obtidos do OpenStreetMap e o Modelo Digital de Elevação (MDE) utilizado no script anterior:

```
# Importar polígonos de favelas para SP (Código IBGE SP: 3550308)
favelas_sp <- read_favela(year = 2022, simplified = F) %>%
  filter(code_muni == 3550308)

limites_favela <- favelas_sp %>%
  filter(code_favela == 35503080692)

limites_sf <- st_bbox(limites_favela)

# Consulta à API Overpass para extrair as vias residenciais
bb <- opq(bbox = limites_sf, timeout = 180, memsize = 268435456)

vias <- bb %>%
  add_osm_feature(key = "highway", value = "residential") %>%
  osmdata_sf()

# Extração das geometrias do tipo linha
linhas_vias <- vias$osm_lines

# Transformação do referencial dos dados para SIRGAS2000 projeção UTM fuso 23S
linhas_vias <- linhas_vias %>%
  st_transform(31983)

# Visualização das vias
mapview(linhas_vias, color = "red")

# Segmentando as vias em trechos de 10 m
segmentos_vias <- line_segment(linhas_vias, segment_length = 30)

# Adicionar identificador sequencial dos segmentos
segmentos_vias$id_segmento <- 1:nrow(segmentos_vias)
```

```r
# Extrair os vértices iniciais e finais de cada segmento
pt_inicio <- st_line_sample(segmentos_vias, sample = 0) %>% st_cast("POINT")
pt_fim    <- st_line_sample(segmentos_vias, sample = 1) %>% st_cast("POINT")


# Amostrar os valores de elevação do MDS para os pontos inicial e final
segmentos_vias$z_inicio <- terra::extract(mds_utm, vect(pt_inicio))[, 2]
segmentos_vias$z_fim    <- terra::extract(mds_utm, vect(pt_fim))[, 2]

# Calcular o comprimento real (em metros) e a declividade (%)
segmentos_vias <- segmentos_vias %>%
  mutate(comprimento_m = as.numeric(st_length(.)),
         delta_z = abs(z_fim - z_inicio),
         declividade_pct = (delta_z / comprimento_m) * 100)

# Visualização dos segmentos com a declividade no mapview
mapview(segmentos_vias, zcol = "declividade_pct", layer.name = "Declividade (%)")
```

### 6.3.4 Reclassificação de dados Raster

A operação de reclassificação de dados raster é utilizada quando se deseja transformar dados contínuos em classes discretas, agrupar valores em intervalos ou substituir valores existentes por novos valores. No exemplo a seguir, o script realiza a reclassificação do raster de declividade, convertendo seus valores contínuos em classes discretas:

```r
# Mapa de declividade (slope) em percentual a partir do MDE
declividade_rad <- terrain(mds_utm, v = "slope", unit = "radians")

# Conversão para percentual: Tan(Declividade) * 100
declividade_perc <- tan(declividade_rad) * 100

# View the percentage slope raster
plot(declividade_perc, main = "Declividade em Percentual (%)")

# Definir a matriz de reclassificação por intervalos de declividade
# Relevo: Plano (0-3%), Suave Ondulado (3-8%), Ondulado (8-20%), Forte Ondulado (20-45%), Mo
matriz_reclass <- matrix(c(
  0,   3, 1,   # Classe 1: Plano
  3,   8, 2,   # Classe 2: Suave Ondulado
  8,  20, 3,   # Classe 3: Ondulado
  20,  45, 4,   # Classe 4: Forte Ondulado
```

```
  45, 75, 5,    # Classe 5: Montanhoso
  75, Inf, 6    # Classe 6: Escarpado
), ncol = 3, byrow = TRUE)

# Reclassificar o raster utilizando a função classify()
declividade_reclass <- classify(declividade_perc, rcl = matriz_reclass, right = FALSE)

# Atribuir rótulos às classes no raster do terra
levels(declividade_reclass) <- data.frame(
  id = 1:6,
  classe = c("Plano (0-3%)", "Suave (3-8%)", "Ondulado (8-20%)", "Forte Ondulado (20-45%)",
)

# Visualizar o resultado da reclassificação
plot(declividade_reclass, col = c("lightgreen", "yellow", "orange", "red", "purple", "black"
```

## 6.4 Referências:


https://r-spatial.github.io/sf

https://cran.r-project.org/web/packages/osmdata/index.html

https://cran.r-project.org/web/packages/terra/refman/terra.html

https://cran.r-project.org/web/packages/geobr/vignettes/intro_to_geobr.html

https://cran.r-project.org/web/packages/exactextractr/refman/exactextractr.html

https://cran.r-project.org/web/packages/elevatr/refman/elevatr.html

https://cran.r-project.org/web/packages/stplanr/index.html

https://cran.r-project.org/web/packages/lidR/index.html

https://cran.r-project.org/web/packages/rgl/index.html

# Referências


BURROUGH, P. A.; MCDONNELL, R. A. Principles of geographical information systems. New York: Oxford University Press, 1998. 334p.

DE SMITH, M. J.; GOODCHILD, M. F.; LONGLEY, P. Geospatial analysis: a comprehensive guide to principles, techniques and software tools. Troubador Publishing Ltd, 2025 - 7th edition. Disponível em: http://www.spatialanalysisonline.com/HTML/index.html

GEMAEL, C. Introdução à Geodésia Física. Editora da UFPR. Curitiba, 1999.

HOFMANN-WELLENHOF, B.; LICHTENEGGER, H.; WASLE, E. GNSS: Global Navigation Satellite Systems – GPS, GLONASS, Galileo & more. Austria: Springer-Verlag Wien New York, 1a edição, 2008. 516p. ISBN 978-3-211-73012-6

LOVELACE, R.; NOWOSAD, J.; MUENCHOW, J.Geocomputation with R. Chapman and Hall/CRC, 2025. Disponível em: https://doi.org/10.1201/9780203730058

MACLACHLAN, A.; DENNETT, A. An Applied Geographic Information Systems and Science Course in R.Journal of Open Source Education, v. 5, n. 50, 2022. Disponível em: https://doi.org/10.21105/jose.00141

MONICO, J. F. G. Posicionamento pelo GNSS: Descrição, fundamentos e aplicações. São Paulo, Editora UNESP, 2007. 476p. ISBN: 978-85-7139-788-0

PEREIRA, R. H.; SARAIVA, M.; HERSZENHUT, D.; BRAGA, C. K. V.; CONWAY, M. W. r5r: rapid realistic routing on multimodal transport networks with r 5 in r.*Findings*, 2021. https://doi.org/10.32866/001c.21262

RICHARDUS, P.; ADLER, R.K. Map Projections for Geodesists, Cartographers and Geographers. North-Holland Publishing Company, 1972.

TEAM, R. Core. R language definition.Vienna, Austria: R foundation for statistical computing, v. 3, n. 1, p. 116, 2000. https://mirror.twds.com.tw/CRAN/doc/FAQ/r-release/R-lang.pdf